DEEP LEARNING – EXAM NOTES

Modules 3, 4 and 5: Regularization, Optimization, CNNs, RNNs

Contents

1 MODULE 3 – REGULARIZATION AND OPTIMIZATION FOR TRAINING DEEP MODELS 3
1.1 3.1 – Parameter Norm Penalties and Related Concepts . . . 3
1.1.1 Parameter Norm Penalties . . . 3
1.1.2 Norm Penalties as Constrained Optimization . . . 3
1.1.3 Regularization and Under-Constrained Problems . . . 3
1.1.4 Dataset Augmentation . . . 4
1.1.5 Noise Robustness . . . 4
1.2 3.2 – Semi-Supervised, Multi-Task, Early Stopping, Dropout, and More . . 4
1.2.1 Semi-Supervised Learning . . . 4
1.2.2 Multi-Task Learning . . . 4
1.2.3 Early Stopping . . . 5
1.2.4 Parameter Tying and Parameter Sharing . . . 5
1.2.5 Sparse Representations . . . 5
1.2.6 Bagging and Other Ensemble Methods . . . 5
1.2.7 Dropout . . . 6
1.2.8 Adversarial Training . . . 6
1.2.9 Tangent Distance, Tangent Prop, and Manifold Tangent Classifier . 6
1.3 3.3 – Challenges in Neural Network Optimization and Basic Algorithms . . 7
1.3.1 Challenges in Neural Network Optimization . . . 7
1.3.2 Basic Optimization Algorithms . . . 7
1.4 3.4 – Parameter Initialization, Adaptive Learning Rates, and Second-Order Methods . . . 8
1.4.1 Parameter Initialization Strategies . . . 8
1.4.2 Algorithms with Adaptive Learning Rates . . . 8
1.4.3 Approximate Second-Order Methods . . . 9
1.4.4 Optimization Strategies and Meta-Algorithms . . . 9

2 MODULE 4 – CONVOLUTIONAL NETWORKS 10
2.1 4.1 – The Convolution Operation, Motivation, and Pooling . . . 10
2.1.1 The Convolution Operation . . . 10
2.1.2 Motivation for Convolution . . . 10
2.1.3 Pooling . . . 10
2.1.4 Convolution and Pooling as an Infinitely Strong Prior . . . 11
2.2 4.2 – Variants of the Basic Convolution Function . . . 11
2.3 4.3 – Efficient Convolution, Random/Unsupervised Features, Neuroscientific Basis . . . 11
2.3.1 Efficient Convolution Algorithms . . . 11
2.3.2 Random or Unsupervised Features . . . 11
2.3.3 Neuroscientific Basis for Convolutional Networks . . . 12
2.4 4.4 – Convolutional Networks and the History of Deep Learning . . . 12

3 MODULE 5 – SEQUENCE MODELING: RECURRENT AND RECURSIVE NETS 13

3.1 5.1 – Unfolding Graphs, RNNs, Bidirectional RNNs, Encoder-Decoder . . . 13
3.1.1 Unfolding Computational Graphs . . . 13
3.1.2 Recurrent Neural Networks (RNNs) . . . 13
3.1.3 Bidirectional RNNs . . . 13
3.1.4 Encoder-Decoder (Sequence-to-Sequence) Architecture . . . 14
3.2 5.2 – Deep RNNs, Recursive Nets, Long-Term Dependencies, Echo State, Leaky Units . . . 14
3.2.1 Deep Recurrent Networks . . . 14
3.2.2 Recursive Neural Networks . . . 14
3.2.3 The Challenge of Long-Term Dependencies . . . 15
3.2.4 Echo State Networks (Reservoir Computing) . . . 15
3.2.5 Leaky Units and Multiple Time Scales . . . 15
3.3 5.3 – LSTM, Gated RNNs, Long-Term Dependency Optimization, Explicit Memory . . . 15
3.3.1 Long Short-Term Memory (LSTM) . . . 15
3.3.2 Gated Recurrent Unit (GRU) . . . 16
3.3.3 Optimization for Long-Term Dependencies . . . 16
3.3.4 Explicit Memory . . . 17

Quick Reference Summary 17

MODULE 3 – REGULARIZATION AND OPTIMIZATION FOR TRAINING DEEP MODELS

3.1 – Parameter Norm Penalties and Related Concepts

Parameter Norm Penalties

- **Regularization** is any modification to a learning algorithm that reduces generalization error but not necessarily training error.
- **Parameter norm penalties** add a penalty term $\Omega(\theta)$ to the objective function:

$$\tilde{J}(\theta; X, y) = J(\theta; X, y) + \alpha\,\Omega(\theta)$$

  where $\alpha \geq 0$ is a hyperparameter controlling the strength of regularization.
- Only **weights** w are typically penalized; **biases** are NOT penalized (biases have less variance and do not need regularization).

L2 Regularization (Weight Decay / Ridge)

$$\Omega(\theta) = \frac{1}{2}\|w\|_2^2$$

- Also called **weight decay** or **ridge regression**.
- Drives weights toward zero but rarely makes them exactly zero.
- Equivalent to MAP estimation with a **Gaussian prior** on weights.
- Update rule: $w \leftarrow (1 - \epsilon\alpha)w - \epsilon\nabla_w J$
- Effect: rescales weight vector by a constant factor each step.

L1 Regularization (Lasso)

$$\Omega(\theta) = \|w\|_1 = \sum_i |w_i|$$

- Also called **lasso** regression.
- Produces **sparse** solutions – many weights become exactly zero.
- Equivalent to MAP estimation with a **Laplace prior** on weights.
- Useful for **feature selection**.

Norm Penalties as Constrained Optimization

- Penalized objective can be reformulated as a **constrained optimization** problem:

$$\min_\theta J(\theta; X, y) \quad \text{subject to} \quad \Omega(\theta) \leq k$$

- Using **KKT conditions**, the Lagrangian is: $\mathcal{L} = J(\theta) + \alpha(\Omega(\theta) - k)$
- **Reprojection approach**: after each gradient step, project weights back onto constraint set.
- Advantage of constrained approach: avoids the need to tune α; directly controls the size of the weight vector.

Regularization and Under-Constrained Problems

- Some models are **under-determined** – many solutions achieve zero training error.

- Regularization helps select a specific solution among many.
- Example: **linear regression** with $n < d$ (fewer examples than features) has infinitely many solutions; L2 penalty selects the minimum-norm solution.
- **Moore-Penrose pseudoinverse** provides the minimum-norm least-squares solution without explicit regularization.
- Regularization allows the model to converge even when the **design matrix** $X^T X$ is singular.

Dataset Augmentation

- **Dataset augmentation**: creating fake but realistic training examples by applying transformations to existing data.
- Effective transformations (for images): **translation**, **rotation**, **flipping**, **scaling**, **color jitter**, **cropping**.
- For **speech recognition**: inject noise, change speed, pitch shifting.
- Must apply *label-preserving* transformations only (e.g., do NOT flip a "6" to make a "9" in digit recognition).
- Acts as an implicit regularizer by increasing effective dataset size.
- **Mixup**: generates new examples by linear interpolation of two training examples and their labels.

Noise Robustness

- Adding **noise to inputs** during training is equivalent to a form of data augmentation.
- For linear models, injecting Gaussian noise to inputs is equivalent to **L2 regularization**.
- Adding noise to **weights**: encourages the model to find regions of parameter space that are flat (robust to small perturbations).
- **Label smoothing**: instead of hard labels (0 or 1), use soft labels (e.g., ϵ/k for wrong classes, $1 - \epsilon + \epsilon/k$ for correct class). Reduces overconfidence.
- **Dropout** (see Section 3.2) can be viewed as adding multiplicative noise to hidden units.

3.2 – Semi-Supervised, Multi-Task, Early Stopping, Dropout, and More

Semi-Supervised Learning

- Uses both **labeled** and **unlabeled** data to improve supervised learning.
- In deep learning: learn a representation $h = f(x)$ such that similar inputs map to similar representations.
- **Generative models** (e.g., VAE, GAN) can model $P(x)$ using unlabeled data, then use learned features for classification.
- **Consistency regularization**: enforce that similar inputs produce similar predictions even under perturbation.

Multi-Task Learning

- Multiple tasks share the same input representation, improving generalization via **shared inductive bias**.
- Tasks that share a common underlying structure benefit most.
- **Hard parameter sharing**: all tasks share hidden layers; each task has its own output layer.

- **Soft parameter sharing**: each task has its own model, but parameters are regularized to be close.
- Benefits: acts as a regularizer, increases effective training data size.

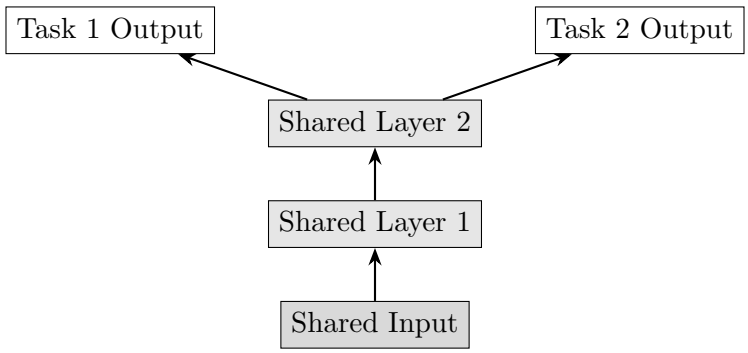


Figure 1: Multi-Task Learning: Hard Parameter Sharing architecture

Early Stopping

- **Early stopping**: halt training when **validation error** starts increasing.
- Most effective and simplest regularization technique in practice.
- Equivalent to **L2 regularization** under certain conditions (e.g., linear models with gradient descent).
- Number of training steps acts as a **hyperparameter** controlling model capacity.
- Procedure: monitor validation loss after each epoch; stop when no improvement for p epochs (**patience**).
- Advantage: requires no modification to the model or loss function.

Parameter Tying and Parameter Sharing

- **Parameter tying**: two models trained on related tasks share (or are penalized to have similar) parameters.
- **Parameter sharing**: exact constraint that certain parameters must be equal.
- Most common application: **Convolutional Neural Networks** – same filter (kernel) applied at every spatial location.
- Reduces number of free parameters dramatically.
- **Siamese networks** use parameter sharing across two input branches.

Sparse Representations

- **Sparsity** can be induced in *representations* (activations), not just parameters.
- Penalty on activations: add $\Omega(h) = \|h\|_1$ to the objective to encourage sparse hidden unit activations.
- Benefits: interpretability, disentangled features, reduced memory.
- **Sparse autoencoders** use this penalty to learn meaningful features.

Bagging and Other Ensemble Methods

- **Bagging** (Bootstrap Aggregating): train k models on k different datasets sampled with replacement from original dataset. Average predictions.

- Reduces **variance** without increasing bias significantly.
- **Ensemble methods**: combine predictions from multiple models to reduce generalization error.
- Each model trained separately; at test time: average (regression) or majority vote (classification).
- Deep learning ensembles are expensive; **dropout** approximates ensemble averaging cheaply.

Dropout

Dropout – Key Facts

- Randomly set each unit to zero with probability p (typically $p = 0.5$ for hidden, $p = 0.2$ for input units) during training.
- At test time: use all units but scale weights by $(1 - p)$ – called **inverted dropout**.
- Equivalent to training an ensemble of 2^n networks (one per possible mask) with weight sharing.
- Prevents **co-adaptation** of neurons.
- Acts as **multiplicative Bernoulli noise** on activations.
- Most effective when the model is large enough to overfit.
- Can be interpreted as Bayesian approximation to deep Gaussian processes.

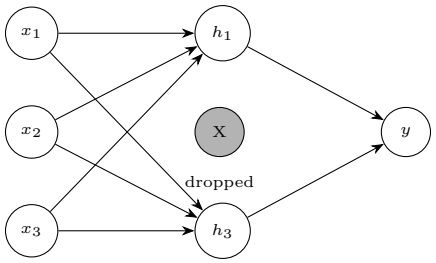


Figure 2: Dropout: crossed unit is dropped (zeroed) during this training step

Adversarial Training

- Neural networks are vulnerable to **adversarial examples**: inputs with small perturbations that fool the network.
- **Adversarial training**: add adversarial examples to the training set to improve robustness.
- **FGSM** (Fast Gradient Sign Method): $x^{adv} = x + \epsilon \cdot \text{sign}(\nabla_x J(\theta, x, y))$
- Acts as a regularizer by reducing sensitivity to small input changes.
- **Virtual adversarial training**: applies adversarial perturbations to unlabeled data for semi-supervised learning.

Tangent Distance, Tangent Prop, and Manifold Tangent Classifier

- **Manifold hypothesis**: natural data lies on a low-dimensional manifold embedded in high-dimensional space.
- **Tangent vectors**: directions along the data manifold (e.g., rotation direction for images).

- **Tangent Distance**: classification metric that measures distance after accounting for transformations (like rotation) that leave class unchanged.
- **Tangent Propagation (TangentProp)**: regularizes the network so that outputs are invariant to known transformations. Adds penalty: $\Omega = \sum_i \left(\frac{\partial f}{\partial x} v^{(i)}\right)^2$ where $v^{(i)}$ are tangent vectors.
- **Manifold Tangent Classifier**: uses an autoencoder to learn the manifold and its tangent vectors, then applies TangentProp automatically without hand-specifying transformations.

3.3 – Challenges in Neural Network Optimization and Basic Algorithms

Challenges in Neural Network Optimization

Key Optimization Challenges

- **Ill-conditioning**: Hessian matrix is poorly conditioned – gradient steps may increase the loss. Gradient norms can grow even as loss decreases.
- **Local Minima**: In practice, most local minima are nearly as good as global minimum (saddle points are more common).
- **Saddle Points**: Gradient is zero but not a minimum in all directions. More problematic in high dimensions.
- **Cliffs and Exploding Gradients**: sharp regions in the loss surface; mitigated by **gradient clipping**.
- **Vanishing Gradients**: gradients become too small to propagate through many layers (especially with sigmoid/tanh).
- **Inexact Gradients**: mini-batch gradient estimates are noisy.
- **Poor Correspondence**: training loss does not always reflect generalization performance.

Basic Optimization Algorithms

Stochastic Gradient Descent (SGD)

$$\theta \leftarrow \theta - \epsilon\, \hat{g}$$

where $\hat{g} = \frac{1}{m} \nabla_\theta \sum_i L(f(x^{(i)}; \theta), y^{(i)})$ is the mini-batch gradient.

- Noisy gradient is computationally cheaper and provides implicit regularization.
- Learning rate ϵ must decay over time: $\epsilon_k = (1 - \alpha)\epsilon_0 + \alpha\epsilon_\tau$ or $\epsilon_k \propto 1/k$.

Momentum

$$v \leftarrow \alpha v - \epsilon \nabla_\theta J(\theta), \qquad \theta \leftarrow \theta + v$$

- $\alpha \in [0, 1)$ controls how quickly past gradients decay; typically $\alpha = 0.9$.
- Accumulates velocity in directions of consistent gradient.
- **Nesterov Momentum**: evaluates gradient at the *anticipated* position: $\nabla_\theta J(\theta + \alpha v)$.
- Faster convergence, especially for ill-conditioned problems.

3.4 – Parameter Initialization, Adaptive Learning Rates, and Second-Order Methods

Parameter Initialization Strategies

- **Symmetry breaking**: all weights must be initialized differently, or all hidden units learn the same function.
- Biases can be set to zero (or small constant); weights must be random.
- **Xavier / Glorot Initialization**: $W \sim U\left[-\frac{\sqrt{6}}{\sqrt{n_{in}+n_{out}}}, \frac{\sqrt{6}}{\sqrt{n_{in}+n_{out}}}\right]$. Maintains variance of activations and gradients across layers (designed for tanh).
- **He / Kaiming Initialization**: $W \sim \mathcal{N}(0, \sqrt{2/n_{in}})$. Designed for **ReLU** activations.
- **Orthogonal initialization**: initialize weight matrices as random orthogonal matrices. Preserves gradient norms.
- Large initial weights: faster training but risk of saturation or exploding gradients.
- Small initial weights: may cause vanishing gradients.

Algorithms with Adaptive Learning Rates

AdaGrad

$$r \leftarrow r + g \odot g, \qquad \theta \leftarrow \theta - \frac{\epsilon}{\delta + \sqrt{r}} \odot g$$

- Accumulates squared gradients r per parameter; parameters with large gradients get smaller updates.
- Good for **sparse** gradients.
- Problem: r only grows, so learning rate shrinks to zero eventually.

RMSProp

$$r \leftarrow \rho r + (1-\rho) g \odot g, \qquad \theta \leftarrow \theta - \frac{\epsilon}{\sqrt{\delta + r}} \odot g$$

- Uses **exponential moving average** of squared gradients ($\rho \approx 0.9$).
- Fixes AdaGrad's learning rate decay problem.
- Works well for **non-stationary** objectives.

Adam (Adaptive Moment Estimation)

$$\begin{aligned} s &\leftarrow \rho_1 s + (1-\rho_1) g \\ r &\leftarrow \rho_2 r + (1-\rho_2) g \odot g \\ \hat{s} &= \frac{s}{1-\rho_1^t}, \quad \hat{r} = \frac{r}{1-\rho_2^t} \quad \text{(bias correction)} \\ \theta &\leftarrow \theta - \frac{\epsilon}{\sqrt{\hat{r}} + \delta} \hat{s} \end{aligned}$$

- Combines **momentum** (first moment s) and **RMSProp** (second moment r).
- Default: $\rho_1 = 0.9$, $\rho_2 = 0.999$, $\epsilon = 10^{-3}$, $\delta = 10^{-8}$.
- Most commonly used optimizer in practice. Robust to hyperparameter choice.

Approximate Second-Order Methods

- **Newton's Method**: $\theta \leftarrow \theta - H^{-1}g$. Scales gradient by inverse Hessian. Exact: too expensive ($O(n^3)$ to invert H).
- **Conjugate Gradients**: avoid computing Hessian by iteratively solving $Hd = g$ using only Hessian-vector products.
- **BFGS / L-BFGS**: quasi-Newton method; approximates H^{-1} using gradient history. L-BFGS stores only m past gradients (memory efficient).
- **Natural Gradient**: preconditions gradient by inverse **Fisher Information Matrix** F^{-1}. Invariant to parameterization.
- **Kfac**: Kronecker-factored approximation to the Fisher matrix; practical for neural networks.

Optimization Strategies and Meta-Algorithms

- **Batch Normalization**: normalize layer inputs to zero mean and unit variance within a mini-batch. Reduces internal covariate shift. Learnable scale γ and shift β. Acts as regularizer.
- **Coordinate Descent**: optimize one parameter at a time while holding others fixed.
- **Polyak Averaging**: maintain a running average of parameters over time: $\hat{\theta} = \frac{1}{t}\sum_{i=1}^{t}\theta^{(i)}$.
- **Curriculum Learning**: present training examples in a meaningful order (easy to hard).
- **Greedy Layer-Wise Pretraining**: pretrain one layer at a time (autoencoders, RBMs), then fine-tune end-to-end.

MODULE 4 – CONVOLUTIONAL NETWORKS

4.1 – The Convolution Operation, Motivation, and Pooling

The Convolution Operation

Discrete Convolution Formula

$$S(i,j) = (I * K)(i,j) = \sum_m \sum_n I(i-m, j-n)\, K(m,n)$$

In practice, CNNs use **cross-correlation** (no kernel flip):

$$S(i,j) = (I \star K)(i,j) = \sum_m \sum_n I(i+m, j+n)\, K(m,n)$$

- I: input (image), K: kernel (filter).
- Output is called a **feature map**.

Motivation for Convolution

- **Sparse interactions (local connectivity)**: each output unit depends on a small region of the input (**receptive field**), not all inputs. Fewer parameters.
- **Parameter sharing**: the same kernel is applied at every spatial location. Dramatically fewer parameters than a fully connected layer.
- **Equivariance to translation**: if the input shifts, the feature map shifts by the same amount. $f(g(x)) = g(f(x))$.
- Together, these properties make CNNs well-suited for spatial data (images, audio, text).

Pooling

- **Pooling**: replaces the output at a location with a summary statistic of nearby outputs.
- **Max Pooling**: $s = \max_{i \in \mathcal{R}} h_i$. Most commonly used.
- **Average Pooling**: $s = \frac{1}{|\mathcal{R}|} \sum_{i \in \mathcal{R}} h_i$.
- **L2 Pooling**: $s = \sqrt{\sum_{i \in \mathcal{R}} h_i^2}$.
- Benefits: (1) provides **approximate invariance** to small translations; (2) reduces spatial dimensions (**downsampling**); (3) reduces memory and computation.
- Pooling stride controls how much the spatial dimension is reduced.

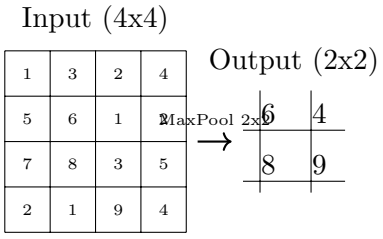


Figure 3: Max Pooling with 2x2 pool size and stride 2

Convolution and Pooling as an Infinitely Strong Prior

- A CNN is equivalent to a fully connected network with an **infinitely strong prior** on its weights.
- The prior says: weights for one hidden unit must be equal to the weights of the spatially adjacent hidden unit (**weight sharing prior**).
- And: weights connecting input to units outside the receptive field must be zero (**locality prior**).
- This prior encodes strong domain knowledge about images: local structure matters and features repeat.

4.2 – Variants of the Basic Convolution Function

- **Stride**: step size when sliding the kernel. Stride $s > 1$ reduces output size and computation.
- **Padding**: add zeros around input borders to control output size.
  - **Valid padding**: no padding; output smaller than input.
  - **Same padding**: pad so output has same spatial size as input.
  - **Full padding**: pad so every input pixel is visited the same number of times.
- **Dilated (Atrous) Convolution**: insert gaps between kernel elements; enlarges receptive field without increasing parameters.
- **1x1 Convolution**: used for channel-wise dimensionality reduction; also called **pointwise convolution**.
- **Depthwise Separable Convolution**: separates spatial and channel filtering; far fewer parameters.
- **Transposed Convolution (Deconvolution)**: used in upsampling / generative models; goes from small feature map back to larger.
- **Structured Outputs**: convolutions can produce 2D output maps for tasks like **semantic segmentation** (pixel-level classification).
- **Data Types**: 1D convolution (sequences/audio), 2D convolution (images), 3D convolution (video/volumetric data).

4.3 – Efficient Convolution, Random/Unsupervised Features, Neuroscientific Basis

Efficient Convolution Algorithms

- Naive convolution: $O(k^2 \cdot n^2)$ for $k \times k$ kernel on $n \times n$ input.
- **FFT-based convolution**: $O(n^2 \log n)$ using the convolution theorem: $f * g = \mathcal{F}^{-1}(\mathcal{F}(f) \cdot \mathcal{F}(g))$. Faster for large kernels.
- **Winograd algorithm**: reduces multiplications for small kernels (e.g., 3x3); commonly used on GPUs.
- **Im2col**: rearranges input patches into a matrix, then uses GEMM (General Matrix Multiplication); standard GPU implementation.

Random or Unsupervised Features

- Not all convolutional features need to be learned by backpropagation.
- **Random filters**: initialize filters randomly and only train the final classifier. Surprisingly effective.

- **Unsupervised pretraining**: learn filters using k-means, sparse coding, RBMs on unlabeled data.
- Reduces need for labeled data; useful in low-data regimes.

Neuroscientific Basis for Convolutional Networks

- Inspired by **Hubel and Wiesel** (1959–1968): discovered two types of cells in visual cortex.
- **Simple cells**: respond to edges at specific orientations and positions (analogous to convolutional units).
- **Complex cells**: respond to edges at specific orientations regardless of position (analogous to pooling units).
- Hierarchical processing: early layers detect edges, later layers detect shapes and objects.
- **Neocognitron** (Fukushima, 1980): first biologically-inspired hierarchical feature detector.
- **LeNet** (LeCun, 1989): first practical CNN trained by backpropagation on handwritten digits.

4.4 – Convolutional Networks and the History of Deep Learning

- **LeNet-5** (1998): 5-layer CNN for digit recognition; established CNN architecture principles.
- **AlexNet** (2012): 8 layers; won ImageNet; demonstrated power of deep CNNs with GPU training, ReLU, dropout.
- **VGG** (2014): very deep networks (16–19 layers) with only 3x3 convolutions.
- **GoogLeNet / Inception** (2014): Inception modules with parallel convolutions of different sizes; 22 layers.
- **ResNet** (2015): introduced **residual (skip) connections** $h = F(x) + x$; enables training of 100+ layer networks.
- **DenseNet**: each layer connected to all subsequent layers; maximum information flow.
- Key enablers of the CNN revolution: (1) large labeled datasets (ImageNet), (2) GPU computing, (3) better activation functions (ReLU), (4) improved regularization (dropout, batch norm).

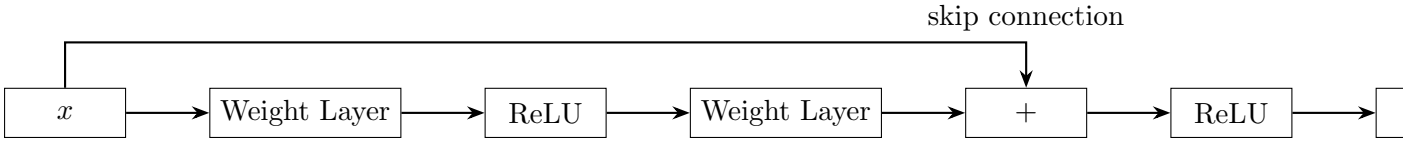


Figure 4: ResNet Residual Block: $h = \text{ReLU}(F(x) + x)$

MODULE 5 – SEQUENCE MODELING: RECURRENT AND RECURSIVE NETS

5.1 – Unfolding Graphs, RNNs, Bidirectional RNNs, Encoder-Decoder

Unfolding Computational Graphs

- A **computational graph** can be unfolded in time to represent a recurrent computation.
- Recurrent relation: $h^{(t)} = f(h^{(t-1)}, x^{(t)}; \theta)$
- Unfolding replaces cycles with a directed acyclic graph (DAG), enabling backpropagation.
- Key insight: **parameter sharing** across time steps – same θ used at every step.
- Length of sequence determines depth of unfolded graph.

Recurrent Neural Networks (RNNs)

Standard RNN Equations

$$
\begin{aligned}
a^{(t)} &= b + Wh^{(t-1)} + Ux^{(t)} \\
h^{(t)} &= \tanh(a^{(t)}) \\
o^{(t)} &= c + Vh^{(t)} \\
\hat{y}^{(t)} &= \text{softmax}(o^{(t)})
\end{aligned}
$$

where W: hidden-to-hidden, U: input-to-hidden, V: hidden-to-output, b, c: biases.

- **Training**: **Backpropagation Through Time (BPTT)** – unroll for T steps, backpropagate through the whole sequence.
- **Teacher Forcing**: during training, feed ground-truth $y^{(t-1)}$ as input at time t (instead of previous prediction). Faster convergence but creates train/test mismatch.
- **Truncated BPTT**: backpropagate only k steps back to save memory and computation.
- Common RNN architectures:
  1. One-to-many (sequence generation)
  2. Many-to-one (sequence classification)
  3. Many-to-many, same length (tagging)
  4. Many-to-many, different length (translation, encoder-decoder)

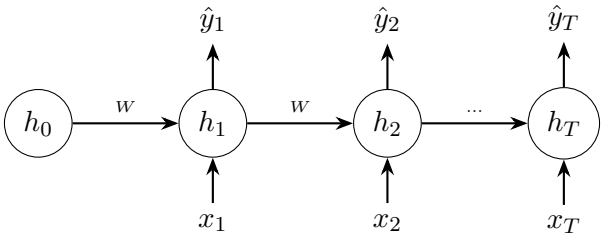


Figure 5: Unfolded RNN: same weights W, U, V at every time step

Bidirectional RNNs

- Standard RNNs process sequence left-to-right; cannot use future context.

- **Bidirectional RNN (BiRNN)**: combines a **forward** RNN (left to right) and a **backward** RNN (right to left).
- At each time step t, output depends on the entire sequence:

$$h^{(t)} = [h^{(t)}_{\text{fwd}}; h^{(t)}_{\text{bwd}}]$$

- Useful for: NLP tasks (POS tagging, NER), speech recognition (offline), protein structure prediction.
- Not applicable to real-time (online) tasks since future is required.

Encoder-Decoder (Sequence-to-Sequence) Architecture

- Maps a variable-length input sequence to a variable-length output sequence.
- **Encoder**: processes input sequence, produces a fixed-length **context vector** $c = f(x_1, \ldots, x_T)$.
- **Decoder**: generates output sequence conditioned on c and previously generated tokens.
- Key limitation: compressing all information into one fixed-size vector – bottleneck for long sequences.
- **Attention mechanism**: allows decoder to focus on different parts of the encoder output at each step. Resolves the bottleneck.
- Applications: machine translation, summarization, speech recognition.

5.2 – Deep RNNs, Recursive Nets, Long-Term Dependencies, Echo State, Leaky Units

Deep Recurrent Networks

- Depth in RNNs can be introduced at multiple points:
  1. Deeper hidden-to-hidden transitions (stack multiple RNN layers).
  2. Deeper input-to-hidden transformations.
  3. Deeper hidden-to-output transformations.
- **Stacked RNNs**: hidden state of layer l at time t is input to layer $l+1$ at time t. More expressive.
- Skip connections across time steps also add a form of depth.

Recursive Neural Networks

- **Recursive Neural Network**: applies the same set of weights recursively over a **tree structure** (not a chain).
- Input: structured data (parse trees, scene graphs).
- Equation: $h^{(t)} = \tanh(Wh_{\text{left}} + Wh_{\text{right}} + b)$ (for binary trees).
- Can model hierarchical composition.
- Applications: sentiment analysis on parse trees, scene understanding.
- Depth scales with tree depth, not sequence length.

The Challenge of Long-Term Dependencies

Vanishing and Exploding Gradients in RNNs

- Gradient at step t involves the product: $\prod_{i=1}^{T} W \cdot \text{diag}(\sigma'(a^{(i)}))$
- If largest singular value of W is < 1: gradient **vanishes exponentially** as T grows.
- If largest singular value of W is > 1: gradient **explodes exponentially**.
- RNNs struggle to learn dependencies across $> \approx 10$ time steps.
- Solutions: **gradient clipping** (for exploding), **LSTM/GRU** (for vanishing).

Echo State Networks (Reservoir Computing)

- **Echo State Network (ESN)**: fix input-to-hidden and hidden-to-hidden weights randomly; only train the **output weights**.
- The fixed recurrent network is called the **reservoir**.
- Key property: reservoir must have the **echo state property** – input history is reflected in current hidden state.
- Efficient to train (linear regression on reservoir states); avoids vanishing gradient entirely.
- Works well for chaotic time series prediction.

Leaky Units and Multiple Time Scales

- **Leaky units**: hidden unit whose activation is a linear combination of previous activation and new input.

$$h^{(t)} = (1-\alpha)h^{(t-1)} + \alpha \tanh(Wh^{(t-1)} + Ux^{(t)} + b)$$

- α controls the **time scale**: small α = long memory; large α = short memory.
- **Multiple time scales**: use a group of leaky units with different α values to capture both fast and slow dynamics.
- Inspired by **clockwork RNN**: partition units into groups, each updating at a different clock rate.

5.3 – LSTM, Gated RNNs, Long-Term Dependency Optimization, Explicit Memory

Long Short-Term Memory (LSTM)

LSTM Equations

$$\begin{aligned}
f^{(t)} &= \sigma(W_f h^{(t-1)} + U_f x^{(t)} + b_f) \quad \textbf{(forget gate)} \\
i^{(t)} &= \sigma(W_i h^{(t-1)} + U_i x^{(t)} + b_i) \quad \textbf{(input gate)} \\
o^{(t)} &= \sigma(W_o h^{(t-1)} + U_o x^{(t)} + b_o) \quad \textbf{(output gate)} \\
\tilde{c}^{(t)} &= \tanh(W_c h^{(t-1)} + U_c x^{(t)} + b_c) \quad \textbf{(cell candidate)} \\
c^{(t)} &= f^{(t)} \odot c^{(t-1)} + i^{(t)} \odot \tilde{c}^{(t)} \quad \textbf{(cell state update)} \\
h^{(t)} &= o^{(t)} \odot \tanh(c^{(t)}) \quad \textbf{(hidden state)}
\end{aligned}$$

- **Cell state** $c^{(t)}$: internal memory; can carry information across many time steps without modification.
- **Forget gate** f: decides what to erase from cell state ($f = 0$: forget, $f = 1$: keep).
- **Input gate** i: decides what new information to write to cell state.
- **Output gate** o: decides what part of cell state to output as hidden state.
- Gradient can flow unchanged through the cell state: **Constant Error Carousel**.
- Introduced by **Hochreiter and Schmidhuber (1997)**.

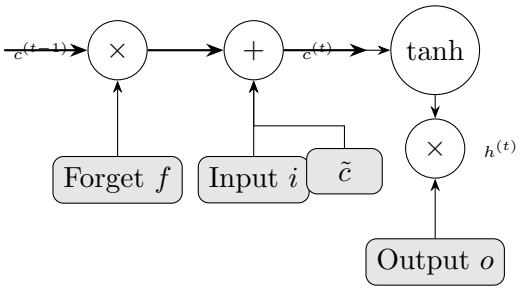


Figure 6: LSTM cell: gates control information flow through the cell state

Gated Recurrent Unit (GRU)

GRU Equations

$$
\begin{aligned}
z^{(t)} &= \sigma(W_z h^{(t-1)} + U_z x^{(t)}) \quad (\textbf{update gate}) \\
r^{(t)} &= \sigma(W_r h^{(t-1)} + U_r x^{(t)}) \quad (\textbf{reset gate}) \\
\tilde{h}^{(t)} &= \tanh(W(r^{(t)} \odot h^{(t-1)}) + U x^{(t)}) \\
h^{(t)} &= (1 - z^{(t)}) \odot h^{(t-1)} + z^{(t)} \odot \tilde{h}^{(t)}
\end{aligned}
$$

- Simpler than LSTM: **no separate cell state**; merges forget and input gates into **update gate** z.
- **Reset gate** r: controls how much previous hidden state is used to compute new candidate.
- Fewer parameters than LSTM; comparable performance in practice.
- Introduced by **Cho et al. (2014)**.

Optimization for Long-Term Dependencies

- **Gradient Clipping**: if $\|\nabla J\| > \tau$, rescale: $g \leftarrow \frac{\tau}{\|g\|} g$. Prevents exploding gradients.
- **Orthogonal / Identity Initialization**: initialize W as orthogonal or identity matrix to prevent early gradient vanishing.
- **Skip connections across time**: add direct connections from $h^{(t-k)}$ to $h^{(t)}$ for faster gradient propagation.
- **Regularize towards identity**: penalize $\|W - I\|^2$ to keep hidden-to-hidden weights near identity.

Explicit Memory

- Standard NNs store knowledge implicitly in weights; weights are updated slowly via gradient descent.
- **External memory**: augment the network with a differentiable memory module that can be read/written.
- **Neural Turing Machine (NTM)**: RNN controller with an external memory matrix. Read/write via soft attention (differentiable).
- **Memory Networks**: multiple memory slots; controller selects relevant memories to answer queries.
- **Attention mechanisms**: a form of soft memory retrieval; allow network to selectively attend to parts of the input sequence.
- **Differentiable Neural Computer (DNC)**: NTM extension with more complex memory addressing.
- Benefit: can store long sequences of facts; not limited by hidden state size.

QUICK REFERENCE SUMMARY

Concept	Key Idea / Formula
L2 Reg	$\tilde{J} = J + \frac{\alpha}{2}\|w\|^2$; weight decay; Gaussian prior
L1 Reg	$\tilde{J} = J + \alpha\|w\|_1$; sparse weights; Laplace prior
Dropout	Randomly zero units ($p = 0.5$); ensemble of 2^n networks
Early Stopping	Stop when val loss rises; equivalent to L2 reg
Adam	Momentum + RMSProp + bias correction; $\rho_1 = 0.9, \rho_2 = 0.999$
Xavier Init	$\text{Var}(w) = \frac{2}{n_{in}+n_{out}}$; for tanh
He Init	$\text{Var}(w) = \frac{2}{n_{in}}$; for ReLU
Convolution	Sparse interactions + parameter sharing + equivariance
Pooling	Max/Average; spatial invariance; downsampling
ResNet	$h = F(x) + x$; skip connections; enables very deep nets
LSTM	Forget, Input, Output gates + Cell state; solves vanishing gradient
GRU	Update + Reset gates; simpler than LSTM
BPTT	Unroll RNN; backprop through time steps
Encoder-Decoder	Encode sequence to context vector; decode to output sequence
Attention	Weighted sum of encoder states; resolves bottleneck
Gradient Clipping	$g \leftarrow \frac{\tau}{\|g\|} g$ when $\|g\| > \tau$; fixes exploding gradients

End of Notes